JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class VolatileCallSite

java.lang.Object
    java.lang.invoke.CallSite
        java.lang.invoke.VolatileCallSite

```
public class VolatileCallSite
extends CallSite
```

A `VolatileCallSite` is a `CallSite` whose target acts like a volatile variable. An `invokedynamic` instruction linked to a `VolatileCallSite` sees updates to its call site target immediately, even if the update occurs in another thread. There may be a performance penalty for such tight coupling between threads.

Unlike `MutableCallSite`, there is no syncAll operation on volatile call sites, since every write to a volatile variable is implicitly synchronized with reader threads.

In other respects, a `VolatileCallSite` is interchangeable with `MutableCallSite`.

See Also:

`MutableCallSite`

Constructor Summary

Constructors

Constructor	Description
`VolatileCallSite(MethodHandle target)`	Creates a call site with a volatile binding to its target.

`VolatileCallSite(MethodType type)`	Creates a call site with a volatile binding to its target.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`MethodHandle`	`dynamicInvoker()`	Produces a method handle equivalent to an invokedynamic instruction which has been linked to this call site.
`MethodHandle`	`getTarget()`	Returns the target method of the call site, which behaves like a `volatile` field of the `VolatileCallSite`.
`void`	`setTarget(MethodHandle newTarget)`	Updates the target method of this call site, as a volatile variable.

Methods inherited from class java.lang.invoke.CallSite

`type`

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Detail

VolatileCallSite

```
public VolatileCallSite(MethodType type)
```

Creates a call site with a volatile binding to its target. The initial target is set to a method handle of the given type which will throw an `IllegalStateException` if called.

Parameters:

`type - the method type that this call site will have`

Throws:

`NullPointerException - if the proposed type is null`

VolatileCallSite

```
public VolatileCallSite(MethodHandle target)
```

Creates a call site with a volatile binding to its target. The target is set to the given value.

Parameters:

`target - the method handle that will be the initial target of the call site`

Throws:

`NullPointerException - if the proposed target is null`

Method Detail

getTarget

```
public final MethodHandle getTarget()
```

Returns the target method of the call site, which behaves like a `volatile` field of the `VolatileCallSite`.

The interactions of `getTarget` with memory are the same as of a read from a `volatile` field.

In particular, the current thread is required to issue a fresh read of the target from memory, and must not fail to see a recent update to the target by another thread.

Specified by:

`getTarget in class CallSite`

Returns:

`the linkage state of this call site, a method handle which can change over time`

See Also:

setTarget(java.lang.invoke.MethodHandle)

setTarget

public void setTarget(MethodHandle newTarget)

Updates the target method of this call site, as a volatile variable. The type of the new target must agree with the type of the old target.

The interactions with memory are the same as of a write to a volatile field. In particular, any threads is guaranteed to see the updated target the next time it calls getTarget.

Specified by:

setTarget in class CallSite

Parameters:

newTarget - the new target

Throws:

NullPointerException - if the proposed new target is null

WrongMethodTypeException - if the proposed new target has a method type that differs from the previous target

See Also:

getTarget()

dynamicInvoker

```
public final MethodHandle dynamicInvoker()
```

Produces a method handle equivalent to an invokedynamic instruction which has been linked to this call site.

This method is equivalent to the following code:

```
MethodHandle getTarget, invoker, result;
getTarget = MethodHandles.publicLookup().bind(this, "getTarget", MethodType
invoker = MethodHandles.exactInvoker(this.type());
result = MethodHandles.foldArguments(invoker, getTarget)
```

Specified by:

`dynamicInvoker` in class `CallSite`

Returns:

a method handle which always invokes this call site's current target

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.